Array 26 (2025) 100406

Contents lists available at ScienceDirect

Array

journal homepage: www.sciencedirect.com/journal/array

A comparative study of machine learning and deep learning models in binary and multiclass classification for intrusion detection systems

Ayesha Alharthi, Meera Alaryani, Sanaa Kaddoura *

College of Technological Innovation, Zayed University, Abu Dhabi, UAE

ARTICLE INFO

Keywords:
Intrusion detection systems
Defensive security
Deep learning
Machine learning

ABSTRACT

Network infrastructure evolution has significantly expanded the attack surface, leading to increasingly complex and sophisticated cybersecurity threats. Traditional rule-based intrusion detection systems (IDS) often fail to detect emerging attack vectors, prompting the need for intelligent, data-driven approaches. This study evaluates and compares the performance of machine learning (ML) and deep learning (DL) models for network intrusion detection. Two publicly available datasets were utilized: a binary-labeled software-defined networking (SDN) dataset and a multiclass industrial control system dataset based on the IEC 60870-5-104 protocol. Preprocessing steps included normalization, label encoding, and a 70:10:20 train-validation-test split. Seven models, Random Forest, Decision Tree, K-Nearest Neighbors, XGBoost, Convolutional Neural Network, Gated Recurrent Unit, and Long Short-Term Memory, were trained and evaluated using precision, recall, and F1-score. The Random Forest model achieved the highest F1-score of 93.57 % on the IEC 60870-5-104 dataset, while XGBoost attained a near-perfect F1-score of 99.97 % on the SDN dataset. These results outperform comparable models in the literature and offer practical insights for selecting effective IDS solutions based on classification type and dataset structure.

1. Introduction

Internet usage has surged recently, bridging communication gaps and facilitating seamless connectivity. However, this rapid expansion has also significantly heightened the risk of cyberattacks, exposing individuals and organizations to new security threats [1]. Cyber-attacks are increasing exponentially yearly, with attackers continuously finding new ways to disrupt systems in critical sectors such as finance and healthcare [2,3]. Distributed Denial of Service (DDoS) attacks are the most common attacks where attackers use multiple machines to overwhelm a target system, eventually rendering it inoperable. As technology evolves, network operations become more automated and rely on software components, minimizing human intervention. This shift necessitates improved network security, particularly against new attack vectors introduced by technologies like Network Function Virtualization [4,5]. As a result, both network services and controllers require advanced protection mechanisms, including Network Intrusion Detection Systems (NIDS). Traditionally, IDS relied on rule-based or signature-based techniques, which often failed to detect novel or evolving threats. Recent advancements have shifted the focus toward machine learning (ML) and deep learning (DL) models, which can learn complex patterns and generalize to unseen attacks. Intrusion Detection Systems (IDS) have evolved significantly over the past decades, from traditional rule-based and signature-based approaches to more adaptive machine learning (ML) and deep learning (DL) methods. While rule-based systems rely on predefined attack patterns, ML and DL models can learn from data to detect novel threats and patterns. This shift has motivated a growing body of research comparing the effectiveness of different learning-based techniques, as outlined in Table 1. Building on this progression, the present study benchmarks classical and deep learning models to guide effective IDS deployment.

Kumar et al. [6] describe two key NIDS techniques: signature-based (SIDS) and anomaly-based (AIDS) detection. SIDS identifies threats by comparing network traffic against known attack signatures [7], AIDS identifies deviations from normal traffic patterns to detect potential anomalies [8]. In the past, attackers needed specialized knowledge of their target systems to launch successful attacks [9]. Today, however, anyone can use widely available tools to exploit networks with minimal expertise, making network security more critical than ever.

Recent advances in Machine Learning (ML), both classical ML models and Deep Learning (DL) techniques, offer promising solutions for intrusion detection systems [10]. DL has proven effective in areas such

* Corresponding author.
E-mail addresses: 202112282@zu.ac.ae (A. Alharthi), 202102492@zu.ac.ae (M. Alaryani), sanaa.kaddoura@zu.ac.ae (S. Kaddoura).

https://doi.org/10.1016/j.array.2025.100406
Received 3 February 2025; Received in revised form 26 April 2025; Accepted 29 April 2025
Available online 8 May 2025

as image recognition, sound processing, and, more recently, intrusion detection [11]. Given the evolving nature of cyber threats, ML and DL models used in NIDS must be trained on comprehensive, up-to-date datasets [12]. These models help detect sophisticated attacks, adapt to new threats, and improve accuracy in identifying malicious activity [13]. However, current ML and DL-based intrusion detection systems research faces significant challenges. Issues such as noise, high dimensionality, class imbalance, inadequate feature selection, and high computational costs have been reported in the literature [14–16]. These limitations hinder the effectiveness of ML and DL models, particularly in detecting new and emerging threats.

Modern intrusion detection systems increasingly face challenges from zero-day attacks, where previously unseen threat patterns evade conventional rule-based and signature-based defenses. To address these issues, recent research has explored few-shot learning and meta-learning techniques, which enable models to recognize new intrusion types with minimal labeled data [17,18]. These approaches are particularly valuable in dynamic and rapidly evolving cybersecurity environments [19]. Although the present study does not directly implement few-shot learning, it comprehensively evaluates traditional and deep learning models across diverse datasets, some of which contain rare classes that partially reflect zero-day attack conditions. The comparative analysis presented can serve as a foundational reference for future work incorporating transfer learning or meta-learning to enhance intrusion detection in real-time scenarios [20,21].

This paper applies several ML models to two widely used datasets, SDN [22] and IEC 60870-5-104 [23,24]. It evaluates their performance in terms of intrusion detection accuracy, bias in prediction, and false

Table 1
Summary of existing literature limitations.

Reference	Proposed Method	Results	Classification Type	Limitations	Applied Approach Advantage
Kumar et al. [25]	Rule-based intrusion detection system using DT	0.848	. Multiclass classification	The reliance on rule-based detection limits the system's ability to identify novel attacks	Utilize different classical ML and DL models for binary and multiclass classification that would learn features from data without being limited to specific rules.
Díaz-Verdejo et al. [26]	Analyzed the effectiveness of SIDS	Up to 0.984 detection rate	Classification type not specified	The study does not propose specific solutions to overcome the identified limitations of SIDS.	
Ahmed [27]	ML with context awareness	SVM: 0.791 SGD: 0.961	Multiclass classification	The approach's applicability to other network types and comparisons with existing methods are not detailed.	Use two datasets collected from different real network environments to assess the performance of models in different network types.
Tonkal et al. [31]	Comparative analysis of several ML models with NCA	KNN: 93.37 DT: 99.85 ANN: 98.10 SVM: 97.70	Binary classification	The study focuses on DL for SDN environments, and its effectiveness in traditional networks and computations remains uncertain.	
Dhanya et al. [32]	Comparative analysis of several ML models	SVM: 0.940 DT: 0.990 RF: 0.990 Adaboost: 0.980 XGBoost: 0.980 MLP: 0.970 KNN: 0.950 MLP: 0.980	Multiclass classification	The computational cost and comparisons with existing methods are not detailed.	Assess the computational cost of different classical ML and DL approaches applied in terms of training time and memory consumption.
Aswathy and Rajkumar [33]	Comparative analysis of several ML models.	DT: 0.840 RF: 0.910 SVM: 0.870 ANN: 0.940 Ensemble: 0.920 DL: 0.950	Classification type not specified.	Discussion on computational cost relied on model types instead of evaluating training time and resource consumption.	
Milosevic et al. [13]	Weighted voting ensemble of deep learning models	0.806	Multiclass classification	The computational complexity of the ensemble approach is not discussed.	
Mozo et al. [29]	ML-based detector for cyberattacks targeting cloud-based SDN controllers	0.995	Classification type not specified.	The computational costs for integrating the ML model with SDN controllers are not addressed.	
Khan et al. [19]	Comparative analysis of several ML models	SVM: 1 KNN: 1 LR: 0.990 GNB: 0.920 CNN: 1 LSTM: 1 Autoencoder: 0.720	Binary classification	The study focused on binary classification, and the computational costs of the algorithms employed are not discussed.	
Mansoor et al. [30]	DL-based approach with SMOTE	0.942	Binary classification	The reliance on simulated data may not fully capture the complexities of real-world network traffic.	The data is kept unbalanced instead of artificially balancing the classes through synthetic data generation, which might reduce the model's performance when applied in real-world cases.
Jeamaon and Khemapatapan [28]	RF	0.969	Multiclass classification	The authors focused on multiclass classification, with the F1-score dropping to 75 % when computing the macro average, showing that the model is not effectively detecting some attacks.	The applied approach does not rely solely on the weighted average F1-score. It evaluates model performance by visualizing the confusion matrix to ensure performance is not biased toward specific classes.

positive rates. In this paper, the approach seeks to identify the best-performing algorithm and compares our results to state-of-the-art methods in the field. The contribution in this paper offers a novel comparison between different algorithms on these datasets and aims to improve the robustness of IDS against modern cyber threats.

This study contributes to the existing body of research by offering a comprehensive comparison of classical machine learning and deep learning models across binary and multiclass intrusion detection tasks. This area remains underexplored in many prior works, which often focus on binary classification alone or evaluate limited model types. By applying a consistent evaluation framework on two diverse and publicly available datasets, one reflecting software-defined networking environments and the other industrial control systems, this work provides insights into the suitability of each model type for varying data structures and attack complexities. The findings highlight high-performing models such as Random Forest and XGBoost and expose the limitations of deep learning models in certain multiclass scenarios. These results offer valuable guidance for researchers and practitioners to design scalable, effective IDS solutions in real-world environments, bridging a gap between theoretical evaluation and practical deployment. The contribution can be summarized as follows:

1. Evaluating multiple models classical and deep learning models under both binary and multiclass scenarios.
2. Compare the performance of the best ML model to state-of-the-art models in the literature.
3. Include both performance and computational cost analysis (training time, CPU, and GPU consumption), which has often been overlooked in previous work.
4. Discussing model behavior with respect to class imbalance, prediction bias, and feature dimensionality provides insights into real-world deployment concerns.
5. Demonstrating that classical ML models, mainly RF and XGBoost, remain highly competitive, often outperforming more complex DL models.

The rest of this paper is divided into multiple sections: Section 2 summarizes the work done related to NIDS; Section 3 presents the methodology used; Section 4 shows the results of the applied models; Section 5 discusses and presents the practical and theoretical implications; Section 7 discusses model limitations and future enhancements; Section 8 concludes.

2. Related work

Various approaches have been proposed in the literature for detecting network attacks, such as DDoS and man-in-the-middle attacks. These methods typically fall into two categories: signature-based and anomaly-based. Signature-based detection identifies attacks by recognizing patterns in the attack traffic, whereas anomaly-based models focus on detecting deviations from normal traffic patterns. These methods are known as rule-based IDS. Yet these rule-based IDS struggle to detect previously unseen attacks. ML models have recently gained prominence since they can effectively learn data features and detect abnormal traffic. Despite their success, these models face challenges such as bias and high false positive rates. ML models applied for IDS can be categorizes as classical ML models and DL models.

2.1. Rule-based IDS

Kumar et al. [25] proposed a rule-based intrusion detection system leveraging decision tree (DT) models to detect attacks based on specific feature sets. They trained four different DT algorithms, C5, chi-squared automatic interaction detection, classification and regression tree, and quick, unbiased, efficient statistical tree, using a dataset of 13 selected features. The authors further demonstrated the rule composition process by combining rules generated from different models. For each model, rules were combined using a logical "OR" operation to produce a comprehensive set of rules for each type of attack. By comparing the rule-based models, the authors highlighted the effectiveness of DT models in generating accurate and reliable rules for intrusion detection, particularly in handling attacks such as DoS. The paper proposed a rule-based intrusion detection system which limits the system's ability to identify novel attacks.

Díaz-Verdejo et al. [26] have studied the performance of three SIDS in detecting web attacks. The assessment was based on the detection rate achieved with predefined subsets of rules for Snort, ModSecurity, and Nemesida using seven distinct attack datasets. Their work addresses a critical aspect of intrusion detection: the effectiveness of predefined rule sets in various operational contexts. The authors emphasized the need for continuous updates with SIDS to enhance detection capabilities, but they did not propose specific solutions to overcome the identified limitations.

2.2. Classical ML-based IDS

Ahmed [27] proposed an intrusion detection model for wireless sensor networks to enhance detection by integrating ML and context-awareness techniques. The approach utilized principal component analysis and singular value decomposition for feature selection, then classifying instances using a Support Vector Machine (SVM) with Stochastic Gradient Descent (SGD). The model achieved an impressive 97 % F1-score in classifying network traffic as normal or malicious. The authors focused on detecting intrusions in wireless sensor networks without comparing their approach with existing methods or showing its applicability to other network types.

Jeamaon and Khemapatapan [28] implemented an intrusion detection system using the CICIDS-2017 dataset, employing a Random Forest (RF) model with hyperparameter tuning. Their model achieved a classification accuracy of 96.94 % and an F1-score of 97.86 %, highlighting the importance of careful tuning for model performance. The authors focused on multiclass classification, with the F1-score dropping to 75 % when computing the macro average, showing that the model is not effectively detecting some attacks.

Mozo et al. [29] conducted a comprehensive study on incorporating ML into a distributed framework to secure telecom networks, particularly in virtual private network services. The study addressed emerging attack vectors, including crypto-mining malware, and integrated Green Artificial Intelligence techniques to minimize the energy consumption of ML models without sacrificing accuracy. Furthermore, the models were fortified against adversarial attacks, ensuring the cybersecurity components of TeraFlowSDN were resilient to sophisticated attack attempts. The computational costs for integrating the ML model with SDN controllers are not addressed.

2.3. DL-based IDS

Milosevic et al. [13] proposed an ensemble model for intrusion detection, leveraging a weighted combination of multiple DL models. Their approach yielded high detection performance, showcasing the potential of ensemble learning for improving accuracy and robustness. Although ensemble models increase complexity, the authors did not discuss the proposed approach's computational complexity and deployment feasibility. Mansoor et al. [30] focused on detecting DDoS attacks in SDN through a DL model. They implemented a combined feature selection method using the information gain ratio and chi-square ($\chi 2$) to enhance feature selection before applying the RNN model, which achieved an F1-score of 94.28 %. The authors used SMOTE for data balancing; however, real-world data is imbalanced, and reliance on simulated data may not fully capture the complexities of real-world network traffic. Additionally, they do not discuss the computational cost of the RNN model.

2.4. Comparative studied of ML and DL for IDS

Khan et al. [19] examined the performance of classical ML and DL models for intrusion detection. They reported that models such as Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), SVM, and KNN achieved 100 % accuracy, suggesting potential overfitting and raising concerns about their generalizability when applied to real-world data. The authors focused on binary classification and did not discuss the computational costs of the algorithms employed, especially the DL models.

Tonkal et al. [31] conducted a comparative analysis of several ML models for detecting DDoS attacks in SDN. They explored classical models, such as k-nearest Neighbors (KNN), DT, and SVM, as well as DL models, like Artificial Neural Networks (ANN) with Neighborhood Component Analysis (NCA) feature extraction methods. The DT model outperformed others with 100 % accuracy. However, this level of accuracy suggests overfitting, which could result in degraded performance on data with different distributions. The effectiveness of the ANN model in other networks remains uncertain as the authors focused only on binary classification for SDN networks.

Dhanya et al. [32] compared the performance of multiple machine learning models, including SVM, DT, RF, Adaboost, XGBoost, Multilayer Perceptron (MLP), and Artificial Neural Network (ANN). The DT model produces the best accuracy of 99.05 % compared to other algorithms. They have found that all features in the dataset are important for model detection since removing features from the dataset dropped the models' performance. The study focused only on the model performance without comparing it with state-of-the-art models or studying the computational cost, especially for the ANN model.

Aswathy and Rajkumar [33] evaluated the performance of various ML models for anomaly detection in network traffic. The models were categorized into classical, DL, and ensemble approaches. Hyperparameter tuning was employed to optimize the models' performance. Their study revealed a trade-off between accuracy, computational efficiency, interpretability, and scalability. While DL models demonstrated robustness in anomaly detection, they demanded significant computational resources. The author relied on discussing computational efficiency based on the model type without evaluating training time and resource consumption.

Previous studies have explored using ML and DL models for network intrusion detection. These techniques have enhanced the ability to detect and classify network intrusions more effectively than traditional approaches. However, there are still several gaps in the implementation of an IDS. Several studies relied on proposing models for either binary [30] or multiclass classification [25] while focusing on a single dataset [13,32]. Additionally, the computational costs associated with these models are often overlooked. Table 1 summarizes the limitations in the existing literature. Unlike most prior research, this work distinguishes itself by applying and comparing multiple ML and DL approaches across binary and multiclass classification tasks using two distinct datasets: an SDN dataset and the IEC 60870-5-104 dataset. This paper also evaluates the models' prediction performance and analyzes their computational efficiency. Another challenge in intrusion detection is noise, high dimensionality, and class imbalance that hinder the performance of ML and DL models. This paper applies data preprocessing techniques to remove noise from real-world datasets (SDN dataset and the IEC 60870-5-104).

Additionally, irrelevant or less significant features are removed during preprocessing to reduce dimensionality. To ensure that models are trained and evaluated in conditions that resemble actual deployment environments, the data is kept unbalanced instead of artificially balancing the classes through synthetic data generation, which might reduce the model performance when applied in real-world cases. Addressing these gaps provides a more representative of real-world scenarios where network traffic encompasses various attack types. Furthermore, this work emphasizes finding the best threat detection model with reduced bias and less dependence on irrelevant features.

3. Methodology

The goal of this study is to evaluate and compare the effectiveness of classical machine learning, deep learning, and large language models in detecting cyber intrusions across both binary and multiclass classification tasks. The problem addressed involves accurately identifying different types of malicious activities from network traffic, with attention to efficiency, performance, and adaptability across varied network environments. To achieve this, we implement a standardized pipeline on two publicly available datasets and assess model performance using key classification metrics.

3.1. Dataset

The binary SDN dataset [22] and the multiclass IEC 60870-5-104 [23,24] are employed to implement NIDS. Both datasets simulate realistic network conditions, incorporating normal and malicious traffic, which is crucial for effective model training and accurate intrusion detection. The datasets used in this study, IEC 60870-5-104 and the SDN dataset, were obtained from publicly available sources. The IEC 60870-5-104 dataset simulates network traffic from industrial control systems (ICS), while the SDN dataset reflects traffic flows in a software-defined networking environment, including various intrusion types. Although these datasets were not collected as part of this study, both are widely used in IDS research and contain labeled records suitable for supervised learning.

The IEC 60870-5-104 and SDN datasets were selected to evaluate different ML models on both binary and multi-class intrusion detection scenarios, which reflect common real-world security needs. The IEC 60870-5-104 dataset is based on a critical industrial protocol widely used in SCADA systems [34]. It contains a multi-class structure with several attack types (e.g., DoS, MITM), allowing us to assess the model's ability to distinguish between different types of threats in an industrial control system environment. The SDN dataset represents a software-defined network infrastructure, which is increasingly being adopted in enterprise and cloud environments. This dataset uses a binary classification format, which is ideal for evaluating whether ML models can detect malicious behavior in more dynamic, programmable network contexts.

The creators of the datasets utilized testbed environments to simulate normal and attack scenarios, including IoT-related traffic in SDN. While this study did not perform additional feature engineering beyond basic preprocessing, it is acknowledged that domain-specific feature extraction, especially in IoT contexts, can significantly enhance model robustness and generalization. The SDN dataset, generated using the Mininet emulator, includes a mix of TCP, UDP, and ICMP traffic, focusing on DDoS attacks. It contains 22 features and a label column, initially comprising 104,345 records.

After preprocessing and removing duplicates and null values, 98,748 samples remained. The dataset includes network features like IP addresses, packet count, byte count, and duration. Switch numbers, source IP addresses, and destination IP addresses were excluded to prevent model bias. These features were removed to avoid overfitting, where the model might incorrectly associate malicious behavior with specific network devices or IPs, mainly if certain switches or IPs predominantly handle malicious traffic. Additionally, time-based features, such as date and time, were omitted as they are unrelated to the traffic type and could distort the learning process. Fig. 1 presents the distribution of labels in the dataset. The data is unbalanced showing 61,002 samples of normal network traffic data and 37,726 showing an attack.

The IEC 60870-5-104 dataset is more comprehensive, incorporating TCP traffic with 83 features and a label column showing various attack types. Generated using CICFlowMeter, the dataset is built from multiple files, each representing network traffic at different timestamps. These

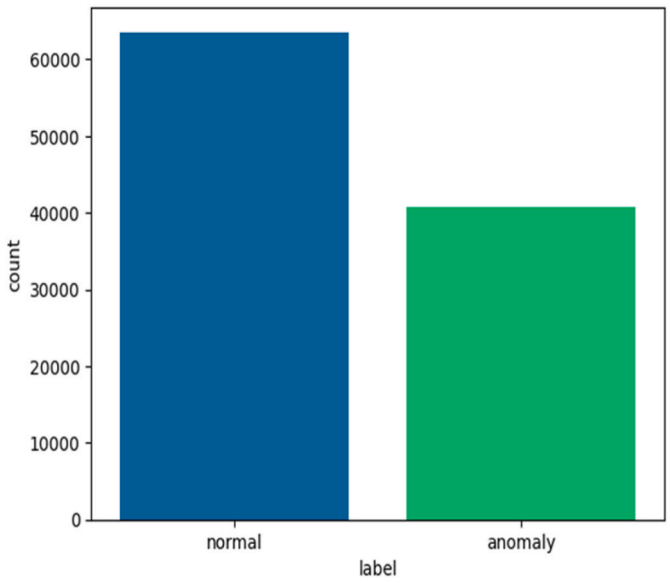

Fig. 1. Label distribution in SDN dataset.

files were combined to create a robust and unbiased dataset, resulting in a final 54,696 instances with no duplicate or null values. Like the SDN dataset, source and destination IP addresses and ports were removed to avoid bias towards specific identifiers, which could skew detection if most normal or malicious traffic were consistently associated with particular IPs or ports. Furthermore, features like flow ID and timestamp were excluded, as they offer no meaningful information for distinguishing between normal and malicious traffic. Fig. 2 presents the distribution of labels in the dataset. Each class in the IEC 60870-5-104 dataset includes 4558 instances.

The SDN dataset shows clear class imbalance, with some attack types appearing far less frequently than others. In contrast, the IEC 60870-5-104 dataset is relatively balanced, allowing for more uniform model evaluation across classes. No specific techniques were applied to address the imbalance in the SDN dataset, such as oversampling, undersampling, or class weighting. This decision was made to assess the baseline performance of the models under naturally imbalanced conditions, reflecting real-world deployment scenarios where rare attack classes often go undetected. While this approach provides insight into model robustness, it is acknowledged that the performance of minority classes in the SDN dataset may have been affected. Future work could explore data augmentation and cost-sensitive learning methods to improve the

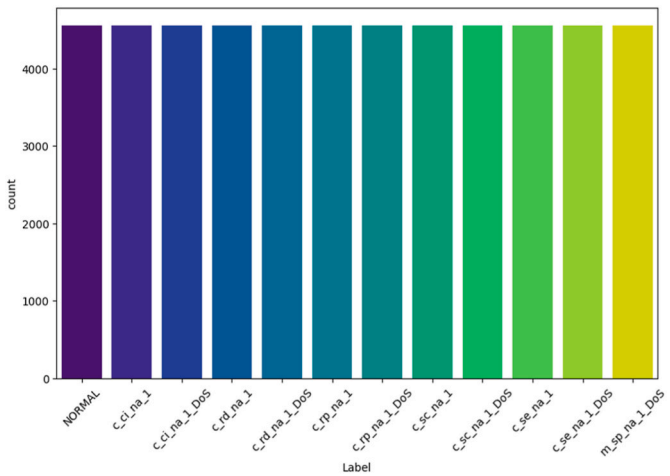

Fig. 2. Label distribution in IEC 60870-5-104 dataset.

detection of underrepresented attack types.

3.2. Intrusion detection approach

The successful implementation of a NIDS hinges on the strategic selection of ML models and meticulous parameter optimization. This study applies to a range of classical ML algorithms and DL models to analyze two distinct datasets: the SDN dataset and the IEC 60870-5-104 datasets. Both datasets are leveraged to detect potential cyber-attacks by analyzing network traffic patterns. The SDN dataset is partitioned into a 70:10:20 ratio, with 65 % allocated for training, 15 % for validation, and 20 % for testing. This split was selected to ensure that a sufficient portion of the data was available for model training (70 %) while also maintaining distinct subsets for model tuning (10 %) and unbiased performance evaluation (20 %). This proportion is commonly adopted in machine learning research, mainly when working with moderately sized datasets, as it provides a good balance between model generalization and the ability to fine-tune hyperparameters. Additionally, using a fixed split across all models ensures a fair and consistent comparison of performance metrics. Fig. 3 illustrates the system model.

The IEC 60870-5-104 dataset already contains separate training, 65 % of the dataset, and testing files, 35 % of the data. The testing file undergoes splitting of 15 % for validation and 20 % for testing. This splitting ensures that, across both datasets, the models are consistently evaluated by 20 % of the data. The validation set is essential for fine-tuning the hyperparameters for each model, ensuring optimal performance and robust detection of attacks.

The preprocessing steps are involved in improving the data representation before training the machine learning models. The numerical feature scaling used is Robust Scaling, which scales data according to quantile range to ensure that all features are on a similar scale and thereby prevent trained models to be biased toward certain features. Label encoding is used to assign integer value for each unique instance in a categorical feature. Encoding allows categorical variables to be included in the model without introducing noise or misleading patterns.

The models selected for this study represent a diverse range of traditional machine learning and deep learning techniques commonly used in intrusion detection research. Random Forest (RF), Decision Tree (DT), K-Nearest Neighbors (KNN), and XGBoost were chosen due to their proven effectiveness in classification problems and their interpretability, which is crucial in cybersecurity applications. On the other hand, deep learning models such as Convolutional Neural Networks (CNN), Gated Recurrent Units (GRU), and Long Short-Term Memory (LSTM) networks were selected for their ability to automatically learn complex patterns and temporal dependencies from data, especially in sequence-based network traffic.

The temporal nature of the datasets was considered during model selection. In particular, the SDN dataset includes network traffic flows recorded over time, allowing for modeling sequential dependencies between data points. This characteristic motivated the inclusion of temporal deep learning models such as LSTM and GRU, which are designed to capture time-dependent patterns and long-range contextual information. These models are particularly suited for scenarios where the timing and order of events, such as the progression of an attack, play a significant role in detection. In contrast, the IEC 60870-5-104 dataset is more structured and does not emphasize sequential attack patterns, which may explain the results presented in the results section and why temporal models underperformed on that dataset.

3.2.1. Random Forest

RF is a commonly used ensemble learning method, particularly effective for classification tasks. This model was selected for its robustness and capability to handle high-dimensional data, making it highly effective for designing NIDS. It builds multiple DTs during training and outputs the classes for predictions. Due to its simplicity, diversity, and ability to mitigate overfitting through the averaging of predictions, it

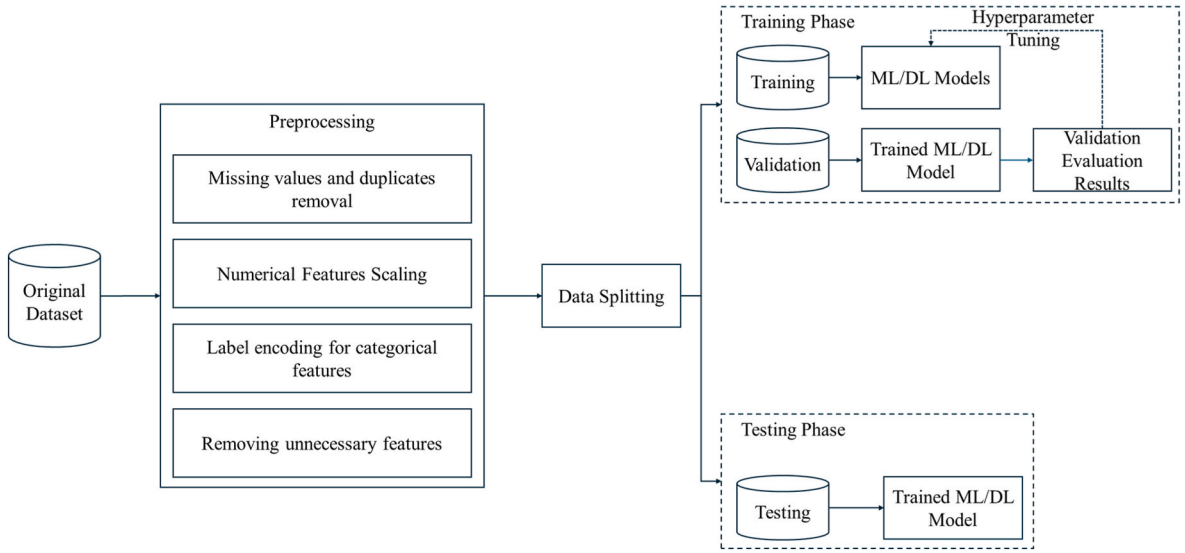

Fig. 3. The system model.

has been shown to deliver high performance in NIDS design [35]. Furthermore, its adaptability to complex network traffic patterns makes it a strong choice for detecting intrusions.

3.2.2. Decision tree (DT)

The DT algorithm was chosen for its intuitive approach and capability to capture descriptive decision-making knowledge directly from the data. It can effectively represent and model the decision-making process of network traffic analysis. Segmenting the data into branches based on feature importance simplifies decision-making, increasing detection accuracy. DTs have demonstrated their ability to take reliable network measures and boost the detection rate, making them a valuable tool in NIDS development [36]. Furthermore, their non-parametric nature allows them to adapt to both linear and nonlinear relationships in data, contributing to their high reliability in intrusion detection scenarios.

3.2.3. K-Nearest Neighbors (KNN)

The KNN algorithm is another key method prominently applied in NIDS due to its strong classification capabilities. KNN classifies new data points based on the majority class among its closest neighbors, which makes it effective for intrusion detection, particularly in recognizing previously unseen attacks. It has shown high detection rates and low false positives, making it suitable for network security applications [37]. Its simplicity in implementation and requiring no prior model training adds to its advantages, although computational costs may increase as the dataset grows.

3.2.4. XGBoost

XGBoost, or Extreme Gradient Boosting, is a DT-based ensemble algorithm with superior performance in various ML tasks, including NIDS. It is well-known for its high accuracy, fast execution, and efficiency in handling large datasets. XGBoost has outperformed other algorithms, such as Gradient Boosting DTs, RF, and SVM, by offering a lower false positive rate, higher detection accuracy, and faster training time [38]. Additionally, its ability to handle missing data and overfitting issues makes it a powerful tool in intrusion detection systems [39].

3.2.5. Convolutional Neural Networks (CNN)

CNN has demonstrated an exceptional ability to automatically extract intricate patterns from complex datasets. In the context of NIDS, CNNs have proven to be highly effective at identifying anomalous behaviors in network traffic and learning spatial hierarchies of data with minimal preprocessing. CNN-based models, especially Deep CNN, have shown a particular advantage in real-time threat detection due to their efficiency in feature extraction and classification [40]. The ability to generalize across different types of attacks and automatically adapt to evolving threats makes CNNs a crucial component of modern intrusion detection systems.

3.2.6. Long Short-Term Memory (LSTM)

LSTM, a variant of recurrent neural networks (RNNs), has been widely adopted in intrusion detection due to its capability to learn long-term dependencies within sequential data. LSTM's ability to retain information over longer time steps allows it to effectively model network traffic patterns and detect intrusions that span over time. LSTM-based NIDS demonstrated remarkable accuracy and detection speed [41]. Additionally, the model's ability to remember important patterns from past network states improves its detection of complex attacks often missed by traditional approaches [42].

3.2.7. Gated Recurrent Unit (GRU)

The GRU is another RNN variant that, like LSTM, can model sequential data with a more simplified architecture, reducing computational overhead. In intrusion detection, GRU has been shown to maintain high accuracy while offering better efficiency, making it suitable for resource-constrained environments such as the Internet of Things (IoT). The D-GRU algorithm, an advanced variant, improves the detection of malicious network traffic by optimizing the number of parameters, making it more practical for deployment in edge devices within the IoT network [43]. Its high accuracy, lower complexity, and reduced resource requirements make it an ideal candidate for real-time intrusion detection in modern network infrastructures.

Appropriate hyperparameter tuning was conducted for each algorithm to ensure fair and optimized performance across all models. Table 2 summarizes the key hyperparameters used for training the classical machine learning and deep learning models evaluated in this study.

3.3. Performance metrics and evaluation

Standard classification metrics like accuracy, precision, recall, and F1 score are used to evaluate the effectiveness of classical ML and DL models. Using these metrics instead of relying solely on accuracy provides a comprehensive assessment of model performance in balanced and imbalanced datasets. The model accuracy presented in (1) measures

Table 2
Hyperparameters used for training machine learning and deep learning models.

Model	Hyperparameters	SDN	IEC
RF	random_state	42	42
	max_samples	2000	None
	n_estimators	100	100
DT	random_state	42	42
	max_features	sqrt	None
	splitter	random	best
	criterion	gini	gini
KNN	n_neighbors	1	1
	p	1	1
XGBoost	random state	42	42
	n_estimators	300	100
	colsample_bytree	0.3	1
	learning_rate	0.02	0.02
CNN	CNN Layers	3	3
	CNN units	128, 64, 32	128, 64, 32
	Maxpooling layers	3	3
	Pool size	2	2
	Dropout layers	4	4
	Dropout rate	0.01, 0.1, 0.1, 0.5	0.01, 0.1, 0.1, 0.5
	optimizer	Adam	Adam
	Learning rate	0.001	0.001
	Batch size	512	512
	epochs	10	10
GRU	GRU layers	3	3
	GRU units	64, 32, 16	64, 32, 16
	Maxpooling layers	2	2
	Pool size	2	2
	Dropout layers	4	4
	Dropout rate	0.01, 0.1, 0.1, 0.5	0.01, 0.1, 0.1, 0.5
	optimizer	Adam	Adam
	Learning rate	0.001	0.001
	Batch size	512	512
	epochs	10	10
LSTM	LSTM layers	2	2
	LSTM units	32, 16	32, 16
	Dropout layers	2	2
	Dropout rate	0.5, 0.5	0.5, 0.5
	optimizer	Adam	Adam
	Learning rate	0.001	0.001
	Batch size	512	512
	epochs	10	10

the overall correctness of the model predictions. The precision presented in (2) measures the correctness of positive predictions. The recall presented in (3) measures the model's ability to identify positive cases. The F1 score presented in (4) is the harmonic mean of precision and recall. In the case of binary classification, the TN is the number of samples where the system correctly detects an attack, FN is the number of samples where the system wrongly classifies normal traffic as an attack, TP is the number of samples where the system correctly identifies normal traffic, FP the number of samples where the system fails to detect an attack classifying it as normal traffic. For multiclass classification, these metrics are computed per class and then averaged. Each class is treated as the "positive" class, while all others are considered "negative." The final metric is typically derived using a weighted average, where each class is weighted based on its frequency in the dataset. The equations for these metrics are as follows:

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\text{recall} = \frac{TP}{TP + FN} \tag{3}$$

$$\text{accuracy} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \tag{4}$$

4. Results

4.1. Comparison

The different supervised ML is trained on both datasets and then tested. The performance comparison of ML algorithms for intrusion detection is evaluated using precision, recall, and F1-score metrics. The results of the employed models on the two datasets are presented in Table 3.

The performance evaluation of different models on the IEC 60870-5-104 and SDN datasets shows notable variations. RF demonstrated strong results on both datasets, achieving a precision of 0.9379, a recall of 0.9355, an F1-score of 0.9357 on the IEC 60870-5-104 dataset, and nearly perfect scores on the SDN dataset with a Precision of 1.0, a recall of 0.9853, and F1-score of 0.9926. Similarly, the DT performed well, with a precision of 0.9354, a recall of 0.9331, an F1-score of 0.9334 on the IEC 60870-5-104 dataset, and excellent results on the SDN dataset, with an F1-score of 0.9990. KNN also delivered high performance on both datasets, with an F1-score of 0.9287 on the IEC 60870-5-104 and 0.9973 on the SDN datasets. In contrast, DL models like CNN and GRU performed poorly on the IEC 60870-5-104 dataset, with an F1-score of 0.5373. However, they excelled on the SDN dataset, achieving an F1-score of 0.9990 for CNN and 0.9599 for GRU. XGBoost also showed strong performance, particularly on the SDN dataset, with a nearly perfect F1-score of 0.9997, while it achieved a slightly lower F1-score of 0.8330 on the IEC 60870-5-104 dataset. LSTM struggled significantly on the IEC 60870-5-104 dataset with an F1-score of just 0.0103 but performed exceptionally well on the SDN dataset with an F1-score of 0.9995. Overall, the SDN dataset consistently produced better results across all models, whereas the IEC 60870-5-104 dataset presented more challenges, particularly for the DL models.

To further evaluate the practicality of the models, this study also measured their computational efficiency during training. Tables 4 and 5 present each model's training time, CPU consumption, and GPU consumption when applied to the SDN and IEC 60870-5-104 datasets, respectively. These metrics provide insights into the resource demands of classical machine learning and deep learning models, critical for assessing their scalability and suitability for real-world deployment in resource-constrained environments.

The training time and resource consumption metrics in Tables 4 and 5 highlight clear distinctions between classical machine learning models and deep learning models regarding computational efficiency. On the SDN dataset, traditional models such as KNN (0.02 s) and Decision Tree (0.09 s) demonstrated extremely low training times and minimal CPU usage, making them suitable for lightweight environments. Similarly, Random Forest offered a favorable balance between performance and efficiency, with just 1.94 s of training time and negligible resource consumption.

In contrast, deep learning models exhibited significantly higher training demands. GRU required the longest training time on the SDN dataset (69.55 s), while CNN consumed the most GPU memory (1.4 GB). LSTM also showed a moderate training duration (36.79 s) with GPU usage of 0.4 GB. These trends remained consistent on the IEC 60870-5-

Table 3
Results of applied ML and DL models on both datasets.

Model	IEC 60870-5-104 Dataset			SDN Dataset		
	Precision	Recall	F1-score	Precision	Recall	F1-score
RF	0.9379	0.9355	0.9357	0.99	0.9853	0.9926
DT	0.9354	0.9331	0.9334	0.9980	0.99	0.9990
KNN	0.9300	0.9287	0.9287	0.9968	0.9978	0.9973
XGBoost	0.8375	0.8324	0.8330	0.9998	0.9996	0.9997
CNN	0.5776	0.5697	0.5539	0.9985	1.0	0.9992
GRU	0.5776	0.5697	0.5539	0.9520	0.9830	0.9673
LSTM	0.0052	0.0728	0.0098	0.9978	0.9978	0.9978

Table 4
Training time and resource consumption for each model on the SDN dataset.

Model	Training time (seconds)	Training CPU consumption	Training GPU consumption
RF	1.94	0.0014 GB	0.0 GB
DT	0.09	0.0047 GB	0.0 GB
KNN	0.02	0.0071 GB	0.0 GB
XGBOOST	14.63	0.0095 GB	0.0 GB
CNN	40.04	0.5800 GB	1.4 GB
GRU	69.55	0.3469 GB	0.4 GB
LSTM	36.79	0.2992 GB	0.4 GB

Table 5
Training time and resource consumption for each model on the IEC 60870-5-104 dataset.

Model	Training time (seconds)	Training CPU consumption	Training GPU consumption
RF	23.43	0.2866 GB	0.0 GB
DT	3.89	0.0116 GB	0.0 GB
KNN	0.04	0.0162 GB	0.0 GB
XGBOOST	47.66	0.0351 GB	0.0 GB
CNN	46.20	0.6675 GB	4.4 GB
GRU	58.78	0.2068 GB	1.2 GB
LSTM	23.18	0.1319 GB	0.4 GB

104 dataset, where CNN and GRU again consumed substantial GPU memory (4.4 GB and 1.2 GB, respectively) and exhibited prolonged training durations (46.20 and 58.78 s). The Random Forest model, while having a longer training time on the IEC dataset (23.43 s), remained computationally efficient compared to deep models.

The results suggest that classical ML models are far more efficient regarding training resources, making them highly practical for deployment in resource-constrained or real-time environments. Meanwhile, although potentially more accurate on certain datasets, deep learning models require more computational resources, highlighting a trade-off between performance and scalability that practitioners must consider.

Figs. 4–10 present the confusion matrices of the applied models on IEC 60870-5-104 dataset. Figs. 4, 5 and 7 show a consistent performance of the RF, DT, and XGBoost models on the "normal" class, with all 1111 instances correctly predicted without bias or false classifications. Thus, the model can identify the normal class accurately without being biased

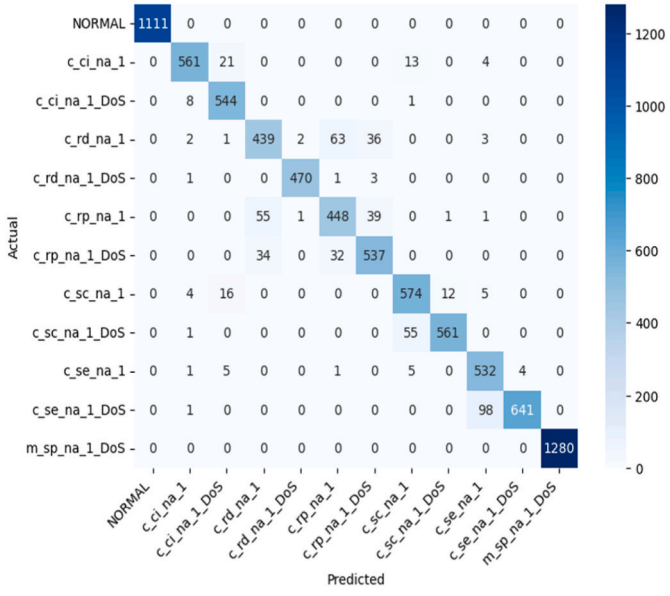

Fig. 4. Confusion matrix for RF model.

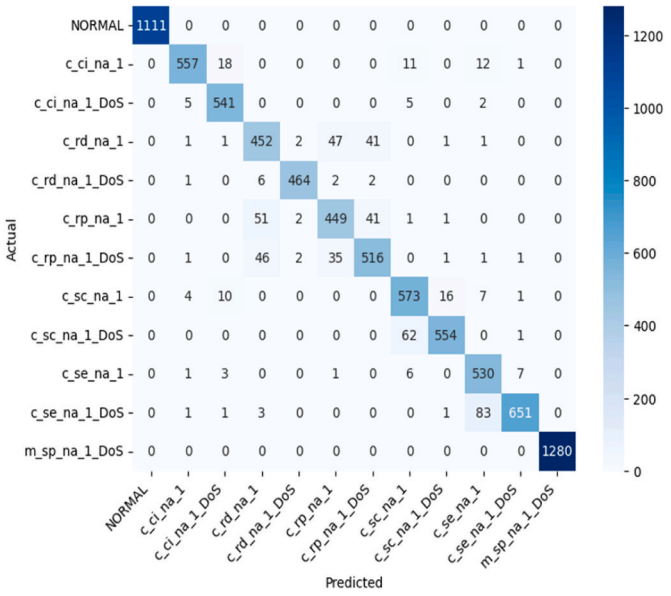

Fig. 5. Confusion matrix for DT model.

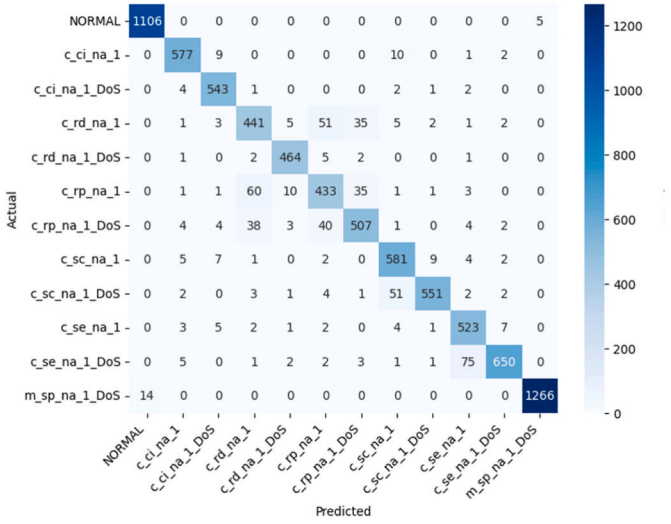

Fig. 6. Confusion matrix for KNN model.

towards attack categories. The models also performed well on the "m_spa_na_1_DoS" attack class, predicting 12,850 instances correctly with no misclassifications across other attack types. Figs. 8 and 9 reveal similar performances of CNN and GRU models across all classes, correctly predicting 1107 instances of the "normal" class. However, both models struggle with distinguishing between certain attack classes.

Despite the overall effectiveness of the models, a closer examination reveals recurring misclassifications between certain attack types. In particular, the models exhibit difficulty distinguishing between classes such as "c_ci_na_1″ and "c_ci_na_1_DoS," likely due to their shared structural patterns and feature similarities, which may result in overlapping decision boundaries. A similar confusion occurs between "c_rp_na_1″ and "c_rp_na_1_DoS," where both models tend to predict instances as "c_rp_na_1," indicating a potential dominance of non-DoS features in both classes or insufficient discriminative features for DoS identification. Additionally, the class "c_sc_na_1″ is often misclassified as "c_sc_na_1_DoS," with nearly half the instances mispredicted, suggesting the presence of feature-level ambiguity or imbalance between the classes. The "c_se_na_1″ and "c_se_na_1_DoS" classes exhibit the same trend,

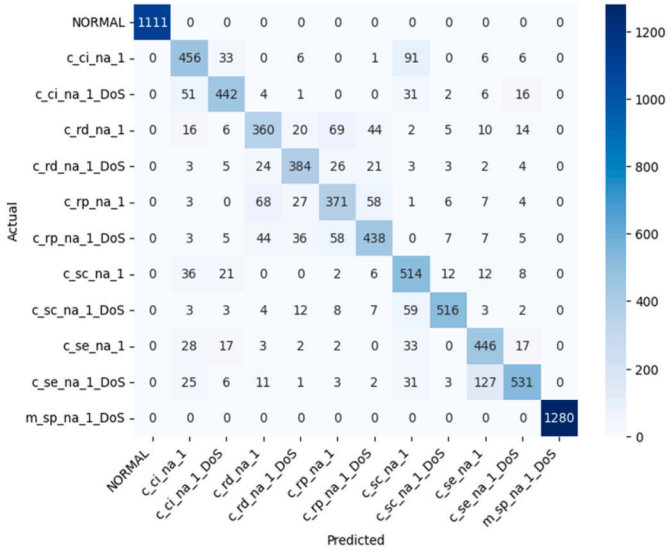

Fig. 7. Confusion matrix for XGBoost model.

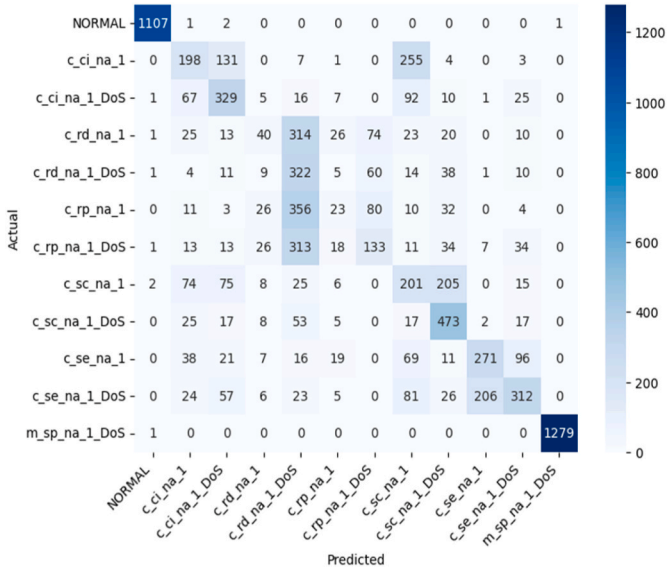

Fig. 8. Confusion matrix for CNN model.

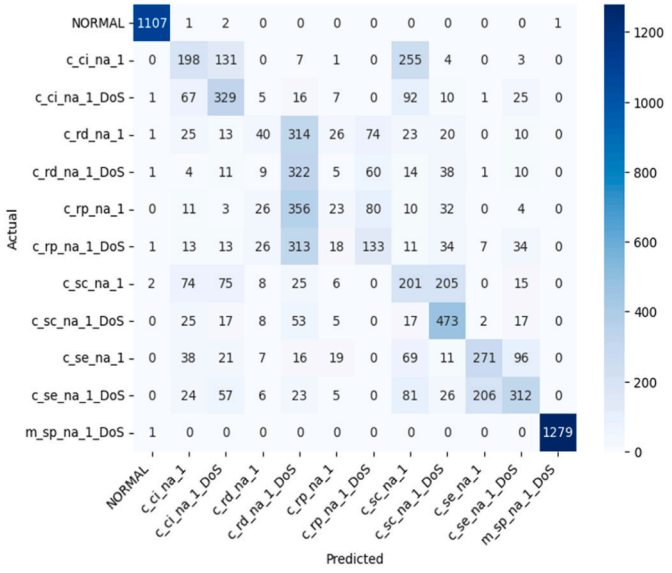

Fig. 9. Confusion matrix for GRU model.

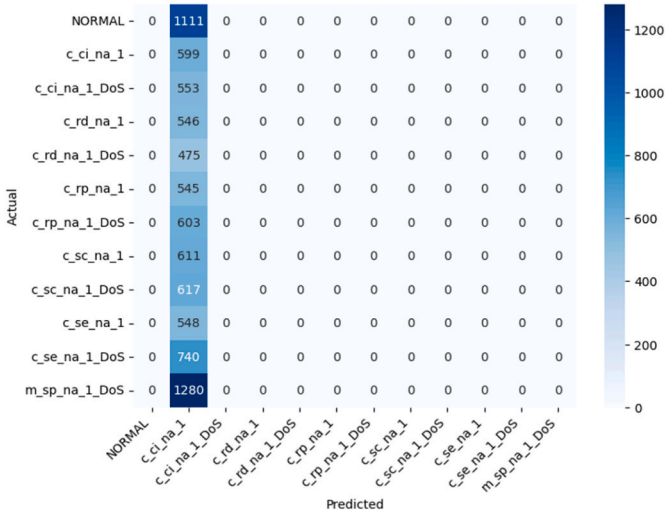

Fig. 10. Confusion matrix for LSTM model.

further supporting that semantically or structurally similar classes with subtle behavioral differences are more challenging for the models to distinguish.

On the other hand, the class "m_sp_na_1_DoS" demonstrates high prediction accuracy, possibly due to its more distinct feature distribution or more explicit attack signature. These findings highlight the need for enhanced feature engineering or class-specific modeling approaches to improve differentiation between closely related attack types. Fig. 10 highlights the poor performance of the LSTM model, which exhibits a strong bias toward the "normal" class. The model fails to correctly predict any other class, with all instances being misclassified as normal.

Figs. 12 and 15 demonstrate that both the DT and CNN models could accurately predict all attack instances without error. Fig. 11 highlights the RF model's ability to correctly classify all "normal" class instances. In Fig. 14, the XGBoost model presents a well-balanced prediction across classes, with only one misclassified instance in the "normal" class and two misclassified attack instances. Figs. 13 and 17 show that the KNN and LSTM models exhibit similar performances. Finally, Fig. 16 reveals that the GRU model performed the worst, reporting the highest false positive and false negative rates, though it still achieved a relatively high overall predictive performance.

4.2. Error analysis

The binary classification results on the SDN dataset demonstrate that tree-based ensemble models, particularly RF and XGBoost, achieved the most balanced and accurate performance across both classes. The per-class precision, recall, F1-score, and error percentages are present in Table 6 for SDN dataset. XGBoost, in particular, achieved the highest overall precision-recall balance, with an F1-score of 0.9947 for the Normal class and 0.9915 for the Attack class, and maintained relatively low error percentages (0.78 % and 0.44 %, respectively). Similarly, RF delivered excellent results, notably outperforming all models in minimizing false negatives on the Attack class, achieving a recall of 0.9968, which is critical for intrusion detection applications where undetected attacks pose significant risks. In contrast, deep learning models such as LSTM and GRU exhibited significantly higher error rates. LSTM, for example, showed a notable drop in precision (0.8436) and F1-score (0.8869) for the Attack class, accompanied by elevated error

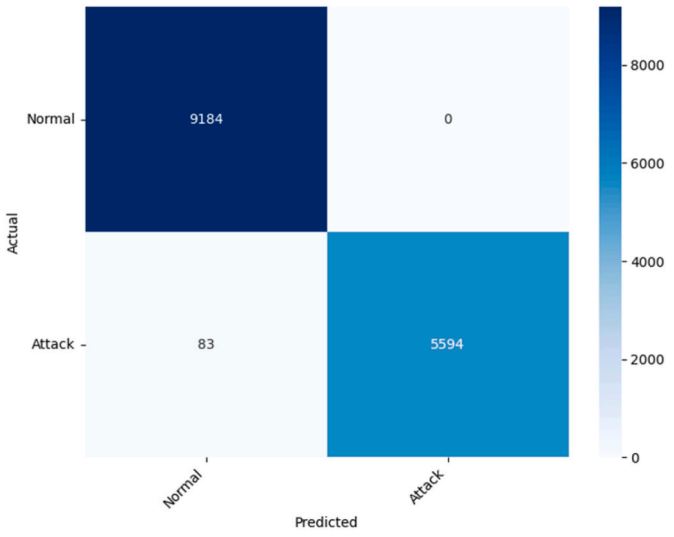

Fig. 11. Confusion matrix for RF model.

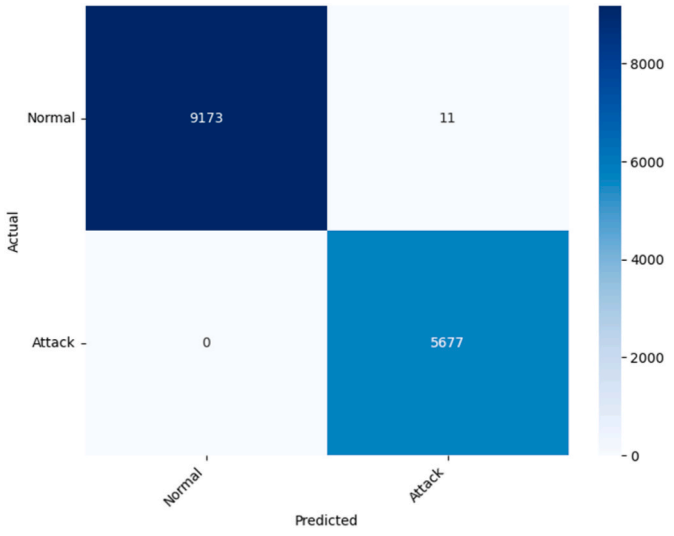

Fig. 12. Confusion matrix for DT model.

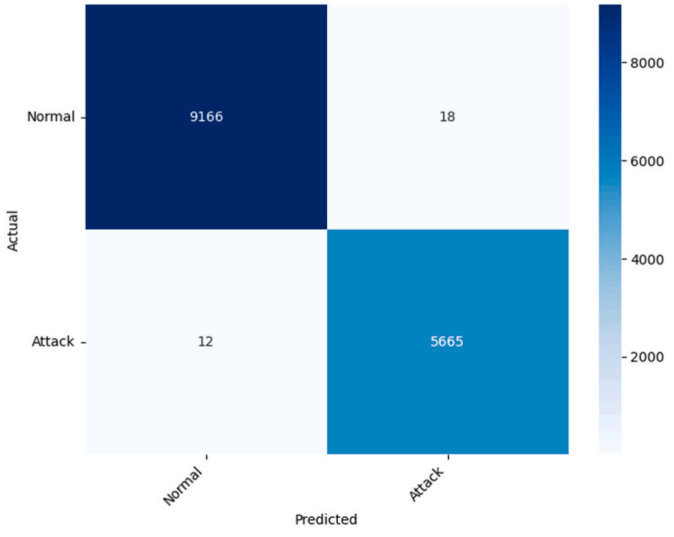

Fig. 13. Confusion matrix for KNN model.

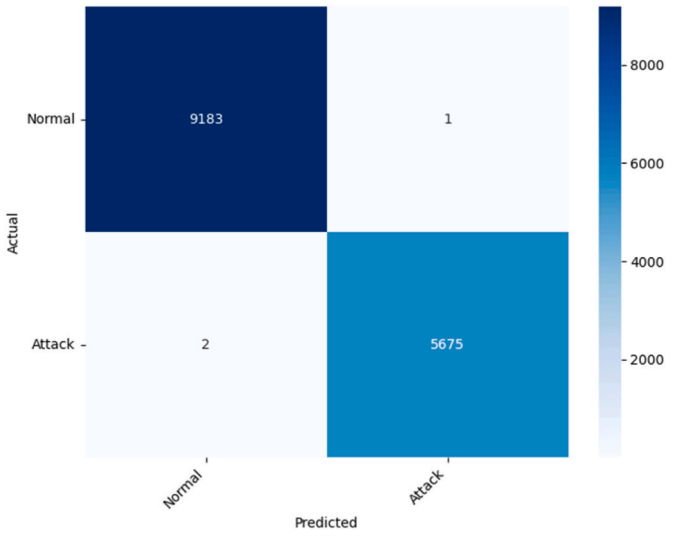

Fig. 14. Confusion matrix for XGBoost model.

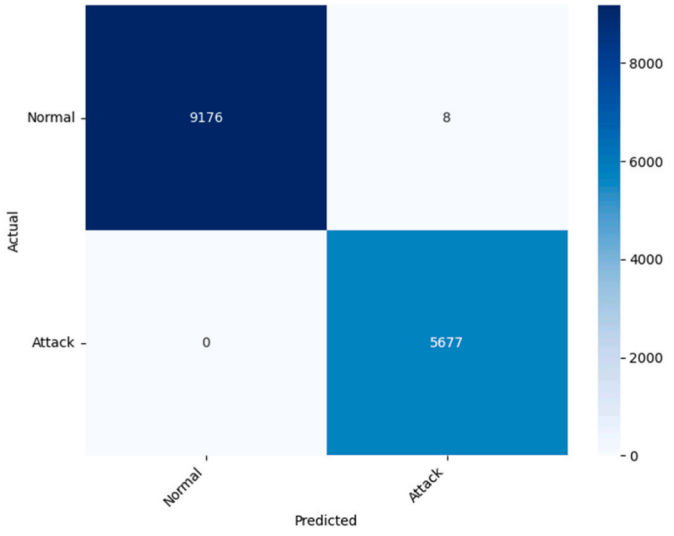

Fig. 15. Confusion matrix for CNN model.

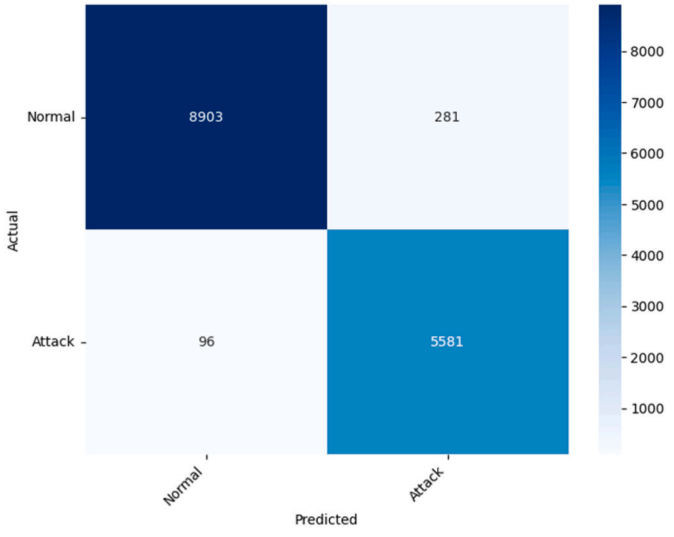

Fig. 16. Confusion matrix for GRU model.

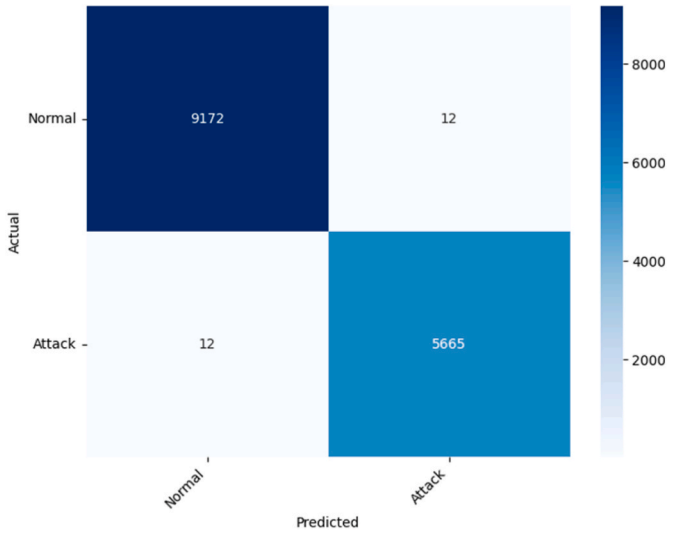

Fig. 17. Confusion matrix for LSTM model.

percentages (10.71 % for Normal, 6.52 % for Attack). These results suggest that LSTM struggled to generalize well to the SDN dataset, potentially due to the data's relatively structured and low-temporal nature, which better suits tree-based models. The analysis reveals that while deep learning models may offer advantages in sequential or temporal data contexts, classical ensemble learners like XGBoost and RF provide more robust and reliable performance for structured network traffic scenarios such as SDN environments.

The multiclass evaluation in Table 7 highlights that classical ensemble models, particularly RF and XGBoost, consistently outperformed other approaches across various attack categories. RF achieved perfect precision, recall, and F1-score for the "Normal" and "m_sp_na_q_DoS" classes and the lowest overall error rates across critical classes such as c_rd_na_1 (5.49 %), c_rp_na_q_1_DoS (13.29 %), c_se_na_1 (15.17 %), and c_sc_na_1_DoS (29.29 %). XGBoost also delivered strong performance, achieving F1-scores above 0.76 for most attack classes, while maintaining relatively low error percentages—for instance, 6.36 % for c_ci_na_1, 14.28 % for c_rd_na_1, and 22.15 % for c_rp_na_1_DoS. In contrast, deep learning models such as LSTM and GRU performed poorly on most classes. LSTM, in particular, had error rates reaching 100 % for several classes (c_ci_na_1_DoS, c_rd_na_1_DoS, c_rp_na_1, c_se_na_1) and near-zero F1-scores, indicating an inability to generalize effectively in this structured and relatively tabular dataset. These models also produced high false negative rates in rare or imbalanced attack classes, making them less suitable for critical infrastructure monitoring. The confusion appears more pronounced in classes with similar characteristics, such as c_rp_na_1_DoS and c_rp_na_1. This suggests that both data characteristics and model architecture influence the performance gap. Tree-based models handle structured, low-dimensional tabular data more effectively, while DL models may require richer temporal or spatial patterns to perform well. In summary, the error analysis supports selecting classical models, particularly Random Forest and XGBoost, for industrial control system intrusion detection scenarios, where structured features dominate and interpretability, precision, and low false negatives are essential.

4.3. Comparison with literature

The comparison of model performance on the IEC 60870-5-104 and SDN datasets reveals that RF and XGBoost are the best-performing models in this study. RF achieved an F1-score of 0.9357 on the IEC 60870-5-104 dataset and 0.9926 on the SDN dataset. XGBoost attained a slightly lower F1-score of 0.8330 on the IEC 60870-5-104 dataset but excelled on the SDN dataset with a near-perfect F1-score of 0.9997. Compared with models from recent literature, RF and XGBoost consistently outperformed them.

Modak and Dehalwar [44] CNN-GRU model achieved lower results on the IEC 60870-5-104 dataset, with an F1-score of 0.5436 and a better but still lower F1-score of 0.9778 on the SDN dataset. Similarly, Gamal et al. [45] presented an LSTM-RNN model that struggled on the IEC 60870-5-104 dataset, with an F1-score of just 0.0098, but reached an F1-score of 1 on the SDN dataset showing overfitting. Mansoor et al. [30] model performed poorly on both datasets, with an F1-score of 0.0521 on the IEC 60870-5-104 and 0.5528 on the SDN datasets. In contrast, the RF and XGBoost models in this study show superior performance, especially on the SDN dataset, highlighting their effectiveness in comparison to models from existing literature.

Table 8 compares the employed models with recent state-of-the-art approaches from the literature. As shown, the Random Forest and XGBoost models outperform previously reported results across both the IEC 60870-5-104 and SDN datasets, particularly in terms of F1-score, highlighting the effectiveness and robustness of the proposed methods.

5. Discussion

The results show that classical ML models, mainly RF, DT, and XGBoost, outperform DL models such as LSTM, GRU, and CNN when applied to IDS datasets. This performance disparity underscores the suitability of classical models for structured/tabular data, where numerical and categorical features dominate. The intrinsic architecture of these classical models, especially their use of decision trees and ensemble methods, allows them to effectively capture patterns in the data through feature importance and interaction terms. Since IDS datasets often contain packet size, protocol type, and duration, which are not temporally or spatially structured, classical models can process these features independently and optimize decision boundaries efficiently. This explains why models like RF and XGBoost achieved near-perfect scores, especially on the SDN dataset, with an F1-score close to 0.9994.

While deep learning models are powerful in capturing complex feature interactions, their performance in multi-class IDS tasks often suffers due to data imbalance and overfitting on majority classes. Dhanya et al. [32] outperformed deep learning models like MLP and ANN in multiclass classification IDS. Additionally, in this paper, the deep learning models underperformed on the IEC 60870-5-104 dataset in the multiclass classification setting. This can be attributed to several factors.

Table 6
Error percentage for SDN dataset.

Model	Precision for Normal	Precision for Attack	Recall for Normal	Recall for Attack	F1 score for Normal	F1 score for Attack	Error Percentage for Normal	Error Percentage for Attack,
RF	0.9980	0.9849	0.9905	0.9968	0.9943	0.9908	0.9472	0.3170
DT	0.9945	0.9891	0.9932	0.9912	0.9939	0.9901	0.6750	0.8807
KNN	0.9888	0.9805	0.9879	0.9819	0.9883	0.9812	1.2086	1.8143
XGBoost	0.9973	0.9874	0.9922	0.9956	0.9947	0.9915	0.7839	0.4403
CNN	0.9927	0.9681	0.9799	0.9884	0.9862	0.9781	2.0143	1.1625
GRU	0.9832	0.9541	0.9710	0.9732	0.9771	0.9636	2.8963	2.6774
LSTM	0.9568	0.8436	0.8929	0.9348	0.9237	0.8869	10.7142	6.5175

Table 7
Error percentage for IEC 60870-5-104 dataset.

Model	RF	DT	KNN	XGBoost	CNN	GRU	LSTM
Precision for Normal	1	1	0.8846	1	0.9811	1	0.0447
Precision for c_ci_na_1	0.6792	0.7433	0.784	0.6901	0.3656	0.4951	0.0041
Precision for c_ci_na_ 1_DoS	0.8804	0.8515	0.8649	0.9101	0.5226	0.548	0
Precision for c_rd_na_1	0.6515	0.6	0.6466	0.6842	0.2088	0.113	0.0447
Precision for c_rd_na_1_DoS	0.9691	0.9474	0.9429	0.824	0.3095	0.3389	0
Precision for c_rp_na_1	0.732	0.7222	0.7315	0.6082	0.2869	0.2684	0
Precision for c_rp_na_1_DoS	0.8253	0.8013	0.7758	0.7834	0.3159	0.3158	0
Precision for c_sc_na_1	0.5691	0.5754	0.6313	0.5389	0.3093	0.3073	0.0994
Precision for c_sc_na_1_DoS	0.9569	0.9375	0.9508	0.8926	0.5119	0.5249	0.0753
Precision for c_se_na_1	0.748	0.7419	0.6923	0.7459	0.5087	0.6339	0
Precision for c_se_na_1_DoS	0.8902	0.869	0.7841	0.8571	0.3579	0.543	0
Precision for m_sp_na_1_DoS	1	0.9951	0.943	1	1	1	0.7195
Recall for Normal	1	0.9949	0.934	1	0.9905	1	0.0614
Recall for c_ci_na_1	0.9172	0.8854	0.8089	0.9363	0.259	0.3167	0.0016
Recall for c_ci_na_ 1_DoS	0.5786	0.6143	0.6857	0.5786	0.6	0.5074	0
Recall for c_rd_na_1	0.9451	0.956	0.9451	0.8571	0.0826	0.0717	0.1326
Recall for c_rd_na_1_DoS	0.6225	0.596	0.6556	0.6821	0.1008	0.5523	0
Recall for c_rp_na_1	0.7474	0.8211	0.8316	0.6211	0.5295	0.2236	0
Recall for c_rp_na_1_DoS	0.8671	0.7658	0.8101	0.7785	0.2174	0.2042	0
Recall for c_sc_na_1	0.7518	0.7518	0.8248	0.708	0.3822	0.4049	0.5986
Recall for c_sc_na_1_DoS	0.707	0.7643	0.7389	0.6879	0.7989	0.8231	0.1229
Recall for c_se_na_1	0.8482	0.8214	0.8036	0.8125	0.487	0.3463	0
Recall for c_se_na_1_DoS	0.7019	0.7019	0.6635	0.6923	0.4092	0.6667	0
Recall for m_sp_na_1_DoS	1	1	0.8878	1	0.9887	1	0.133
F1 score for Normal	1	0.9975	0.9086	1	0.9857	1	0.0518
F1 score for c_ci_na_1	0.7805	0.8081	0.7962	0.7946	0.3032	0.3863	0.023
F1 score for c_ci_na_ 1_DoS	0.6983	0.7137	0.7649	0.7074	0.5586	0.5269	0
F1 score for c_rd_na_1	0.7713	0.7373	0.7679	0.761	0.1184	0.0878	0.0894
F1 score for c_rd_na_1_DoS	0.7581	0.7317	0.7734	0.7464	0.152	0.42	0
F1 score for c_rp_na_1	0.7396	0.7685	0.7783	0.6146	0.3721	0.244	0
F1 score for c_rp_na_1_DoS	0.8457	0.7832	0.7926	0.781	0.2576	0.248	0
F1 score for c_sc_na_1	0.6478	0.6519	0.7152	0.612	0.3419	0.3494	0.1704
F1 score for c_sc_na_1_DoS	0.8132	0.8421	0.8315	0.777	0.624	0.641	0.0934
F1 score for c_se_na_1	0.795	0.7797	0.7438	0.7778	0.4976	0.4479	0
F1 score for c_se_na_1_DoS	0.7849	0.7766	0.7188	0.766	0.3818	0.5985	0
F1 score for m_sp_na_1_DoS	1	0.9976	0.9146	1	0.9943	1	0.2245
Percentage error for Normal	0	0. 5076	6.5989	0	0. 9549	0	93.8608
Percentage error for c_ci_na_1	8.2802	11.4649	19.1082	6.3694	74.1029	68.3307	99.8439
Percentage error for c_ci_na_ 1_DoS	42.1428	38.5714	31.4285	42.1428	40	49.2592	100
Percentage error for c_rd_na_1	5.4945	4.3956	5.4945	14.2857	91.7391	92.826	86.7391
Percentage error for c_rd_na_1_DoS	37.7483	40.3973	34.437	31.788	89.9224	44.7674	100
Percentage error for c_rp_na_1	25.2631	17.8947	16.8421	37.8947	47.0464	77.6371	100
Percentage error for c_rp_na_1_DoS	13.2911	23.4177	18.9873	22.1518	78.2608	79.5841	100
Percentage error for c_sc_na_1	24.8175	24.8175	17.5182	29.197	61.7801	59.5113	40.1396
Percentage error for c_sc_na_1_DoS	29.2993	23.5668	26.1146	31.2101	20.1117	17.6908	87.7094
Percentage error for c_se_na_1	15.1785	17.8571	19.6428	18.75	51.2962	65.3703	100
Percentage error for c_se_na_1_DoS	29.8076	29.8076	33.6538	30.7692	59.0759	33.3333	100
Percentage error for m_sp_na_1_DoS	0	0	11.2195	0	1.1273	0	86.6967

Table 8
Comparison with the literature.

Reference	Model	IEC 60870-5-104 Dataset			SDN Dataset		
		Precision	Recall	F1-score	Precision	Recall	F1-score
Modak and Dehalwar [44]	CNN-GRU	0.5169	0.5386	0.5114	0.9711	0.9767	0.9739
Gamal et al. [45]	LSTM-RNN	0.0052	0.0728	0.0098	1.0	1.0	1.0
Mansoor et al. [30]	IGR-χ2-RNN	0.1593	0.1607	0.0521	0.3820	1.0	0.5528
Employed models	RF	0.9379	0.9355	0.9357	1.0	0.9853	0.9926
	XGBoost	0.8375	0.8324	0.8330	0.9998	0.9996	0.9997

First, deep learning models typically require large and balanced datasets to generalize well, and the IEC 60870-5-104 dataset may not offer sufficient samples or diversity across all classes. Second, the nature of the data might lack the spatial or sequential features that CNNs and RNNs are designed to capture. Unlike the SDN dataset, which benefited from temporal correlations that GRU and LSTM could exploit, the IEC dataset may be more effectively modeled using decision-tree-based methods that perform well on tabular, structured data. Moreover, deep learning models' hyperparameter sensitivity and training complexity could lead to suboptimal performance without extensive tuning, often infeasible in real-time IDS deployments.

Also, the lower performance of DL models in this context can be attributed to their design, which is more suited for processing unstructured data, such as time series or images. Models like LSTM and GRU excel in capturing sequential dependencies, but the IDS datasets used here do not exhibit significant temporal relationships between instances. Similarly, CNNs are highly effective in detecting spatial patterns, which are irrelevant in the context of IDS data. This misalignment between model design and data structure leads to suboptimal performance. LSTM, for example, registered an F1-score of 0 on the SDN dataset,

highlighting the model's inability to leverage its sequential processing power when applied to non-sequential data. DL models generally require large datasets to avoid overfitting and learning meaningful patterns. However, IDS datasets often lack the volume and complexity necessary to justify the use of such models.

The performance differences observed between models, particularly in the case of LSTM and GRU, can be partially attributed to the temporal characteristics of the datasets. The SDN dataset captures network traffic flows over time, enabling sequential models to leverage temporal dependencies for improved classification. This is reflected in the relatively strong performance of GRU and LSTM on the SDN dataset. In contrast, the IEC 60870-5-104 dataset is more tabular and lacks prominent time-based patterns, which may explain the underperformance of temporal models on that dataset. These results highlight the importance of aligning model architecture with the underlying data structure and suggest that temporal models are more suitable for datasets where the sequence and timing of events influence attack detection.

Another crucial factor is the role of feature engineering. Classical models benefit significantly from explicit feature engineering, where domain knowledge is utilized to generate meaningful features from the raw dataset. In IDS tasks, well-engineered features can effectively differentiate between benign and malicious traffic, enhancing the performance of simpler models without requiring the depth and complexity of neural networks. On the other hand, DL models rely on automatic feature extraction, which is advantageous for unstructured data but redundant for structured data like IDS, where human-crafted features tend to be more effective.

Classical models like RF and DT are more accessible to implement and require less hyperparameter tuning than DL models, necessitating extensive optimization of parameters such as learning rates, batch sizes, and layer depths. DL models require significant computational resources and time for fine-tuning, making them less practical for this application, notably when the dataset lacks the size or complexity to justify such overhead.

While DL models have demonstrated remarkable success in domains such as natural language processing and computer vision, their complexity and data requirements make them less effective for structured datasets like IDS. Classical ML models, with their ability to handle structured data, robustness to overfitting, and reliance on domain knowledge, remain the preferred choice for IDS tasks, as demonstrated by their superior performance in this study.

It is crucial to acknowledge that the SDN dataset used in this study exhibits class imbalance, while the IEC 60870-5-104 dataset is balanced. No synthetic resampling techniques (e.g., SMOTE or undersampling) were applied to artificially balance the data to reflect real-world deployment scenarios. Instead, the models were trained on the natural distribution of classes to assess their robustness in realistic conditions where certain attack types are underrepresented. This design choice helps preserve the actual characteristics of network traffic and avoids introducing potential bias from synthetic data. Performance was evaluated using class-sensitive metrics (precision, recall, F1-score) to ensure a fair assessment across majority and minority classes. This contrasts with many previous studies, omitting class imbalance handling or relying on balanced synthetic data, making this study more aligned with practical deployment challenges.

Deploying intrusion detection systems in real-world environments requires careful consideration of false positives and negatives, as both can significantly affect operational efficiency and security. Models with lower precision tend to produce more false positives, which can overwhelm security analysts with alerts and lead to alert fatigue, potentially causing real threats to be overlooked. On the other hand, models with lower recall are prone to false negatives, allowing actual attacks to go undetected, which poses a serious security risk. For instance, in the IEC 60870-5-104 dataset, deep learning models such as GRU and CNN showed relatively low recall and F1-scores, suggesting a higher likelihood of missed intrusions. In contrast, models like Random Forest and Decision Tree demonstrated high precision and recall across both datasets, indicating more balanced performance with lower FP and FN rates. These findings highlight the importance of selecting models based on accuracy and their ability to minimize critical errors, especially in high-risk environments such as industrial control systems or cloud infrastructures. In practical deployments, balancing false alarms and missed detections is essential for maintaining trust in the system and effective threat response.

The findings of this study have significant practical and theoretical implications for NIDS. Practically, the superior performance of classical ML models like RF and XGBoost, especially in terms of lower false positive rates and absence of bias, suggests they are better suited for real-world applications. These models are more computationally efficient and easier to deploy than DL models, making them ideal for organizations with limited resources or the need for real-time detection. Their robustness, particularly on small or imbalanced datasets, further highlights their practicality in cybersecurity environments. Theoretically, the study challenges the assumption that DL models consistently outperform classical approaches, showing that simpler models can generalize better in specific structured data contexts like IDS. The lack of bias in classical models compared to DL also underscores the need to address overfitting and biases in high-dimensional spaces in future research.

Feature dimensionality plays a crucial role in the performance and generalization capability of machine learning and deep learning models. Deep learning models such as CNN, GRU, and LSTM are more susceptible to overfitting when trained on high-dimensional data, particularly without a sufficiently large and diverse dataset. This may partially explain their relatively lower performance on the IEC 60870-5-104 dataset, where the feature space may not offer enough distinct patterns to justify the complexity of these models. In contrast, classical models like Random Forest and Decision Tree, which are inherently more interpretable and less sensitive to high dimensionality, performed consistently well across both datasets. These models also include built-in feature selection mechanisms that help mitigate the impact of irrelevant or redundant features. While deep learning models offer advantages in capturing complex relationships in data, they require careful regularization, dimensionality reduction, or feature engineering to prevent overfitting. Future work may explore techniques such as autoencoders or principal component analysis (PCA) to reduce dimensionality and enhance model performance in deep learning settings.

Many previous studies in the intrusion detection domain acknowledge the challenges posed by noisy data, high dimensionality, and class imbalance but often focus on a single model or dataset. Some work in the literature study only binary datasets, such as Tonkal et al. [31], Khan et al. [19], and Mansoor et al. [30], whereas other study multiclass datasets, such as Jeamaon and Khemapatapan [28], Kumar et al. [25], Ahmed [27], and Milosevic et al. [13]. This work contributes to the literature by systematically comparing multiple machine learning and deep learning models across two different datasets with varied characteristics. By evaluating both binary and multiclass classification performance, the reader can observe how different models handle noise and class imbalance in practice. For example, due to their ensemble nature and inherent feature selection mechanisms, traditional models like Random Forest and Decision Tree showed robustness to noise and class imbalance, particularly in the IEC 60870-5-104 dataset. Moreover, including deep learning models such as GRU and LSTM helped assess the impact of data dimensionality and sequence learning on performance. Unlike many prior works that only use accuracy as a metric, we report precision, recall, and F1-score to better understand model behavior in imbalanced settings. This comparative evaluation fills a gap in the literature by offering a broader view of how these challenges manifest and are mitigated (or not) across model families and dataset types.

Although the IEC 60870-5-104 and SDN datasets used in this study are widely adopted in intrusion detection research, they have limitations. Both datasets were generated in controlled environments, which

may introduce bias due to the lack of real-world network noise or the overrepresentation of specific attack patterns. This could limit the models' ability to generalize when exposed to more diverse and unpredictable traffic in operational settings. Furthermore, the class imbalance in the SDN dataset may lead to biased learning, where models favor majority classes over rare but critical attacks.

Regarding generalizability, while the results demonstrate promising performance on the selected datasets, the effectiveness of these models in real-world scenarios, such as enterprise networks, cloud infrastructures, or IoT ecosystems, requires further validation on live or more heterogeneous datasets. Scalability is another important consideration. Though effective in smaller settings, traditional machine learning models like Random Forest and KNN may become computationally expensive when deployed in large-scale, high-velocity environments. While more scalable in theory, deep learning models often require substantial computational resources and tuning. Future work should investigate the deployment of lightweight or distributed architectures and real-time evaluation frameworks to ensure that the proposed solutions remain practical and efficient under production-level workloads.

The findings of this study hold significant practical value for enhancing real-world cybersecurity infrastructures. In enterprise networks, where large-scale data traffic must be monitored continuously, AI-powered intrusion detection systems can detect sophisticated and evolving threats in real-time, minimizing potential data breaches and financial losses. In cloud computing environments, where dynamic scaling and resource sharing introduce new attack surfaces, machine learning-based IDS can adaptively learn from traffic patterns and detect anomalies that traditional systems may overlook. Similarly, in IoT networks, where devices often operate with limited resources and may lack built-in security mechanisms, lightweight AI models can be deployed at the edge to detect and respond to intrusions promptly. By comparing binary and multiclass classification approaches, the results provide insights into selecting the appropriate model for specific scenarios, such as using binary classification for quick intrusion flagging in resource-constrained environments or multiclass classification in systems requiring detailed threat categorization for proactive defense strategies.

6. Model limitations and future enhancements

While the evaluated models demonstrated strong performance in detecting intrusions across both datasets, several limitations must be considered before real-world deployment. First, deep learning models such as CNN, GRU, and LSTM, although powerful, are computationally intensive and may not be suitable for real-time detection in resource-constrained environments. In contrast, traditional models like RF and KNN, while less demanding, may face scalability issues with large, high-velocity data streams. Another challenge lies in cyber threats' dynamic and evolving nature, which can render static models less effective over time. Additionally, specific models may struggle with imbalanced datasets, leading to biased predictions toward dominant classes.

To enhance the robustness and adaptability of intrusion detection systems, future work should consider integrating hybrid models that combine the high accuracy of deep learning with the efficiency of machine learning techniques. Moreover, advanced feature engineering could improve performance and speed, including automated feature selection and dimensionality reduction methods like PCA or mutual information-based techniques. Continual learning approaches and online learning frameworks may also be explored to enable models to adapt to evolving threats in real-time environments. Finally, testing models on larger, more diverse datasets and simulating deployment in actual network environments will be critical steps toward achieving practical, scalable, and resilient IDS solutions.

Future research should explore the development of hybrid and ensemble models that integrate deep learning with classical machine learning algorithms to capitalize on the strengths of both approaches. Such combinations can improve detection accuracy and efficiency, particularly in multiclass classification tasks, where traditional models may excel in interpretability and deep models in learning complex patterns. Enhanced feature engineering techniques, such as using autoencoders or attention mechanisms for dimensionality reduction and representation learning, can also help reduce computational overhead without sacrificing performance.

Moreover, implementing adaptive learning mechanisms is critical for enabling intrusion detection systems to respond to emerging threats, including zero-day attacks, by continuously updating model knowledge based on new data. Another promising direction is incorporating explainable AI (XAI) techniques to improve model transparency. XAI can provide actionable insights into model decisions, helping cybersecurity professionals understand, validate, and trust automated predictions, especially in high-stakes environments. Finally, evaluating these models across a broader range of heterogeneous and real-world datasets will ensure robustness and generalizability in diverse cybersecurity contexts.

7. Conclusion and future work

This paper comprehensively evaluates classical machine learning and deep learning algorithms for network intrusion detection systems (NIDS), comparing their performance across multiclass and binary classification tasks. The findings reveal that model effectiveness highly depends on the dataset's characteristics. Deep learning models such as CNN performed well on the SDN dataset. In contrast, classical models—particularly Random Forest (F1-score: 93.57 % on the IEC 60870-5-104 dataset) and XGBoost (F1-score: 99.97 % on the SDN dataset), demonstrated robust and, in some cases, superior performance. These results reinforce the practical relevance of classical ML models, especially when computational efficiency, ease of implementation, and interpretability are critical. Beyond benchmark performance, the implications for real-world deployment are significant. In constrained environments with limited computational resources (e.g., edge devices, legacy systems), lightweight models like Random Forest or XGBoost offer a viable and often optimal solution. Conversely, deep learning architectures like CNN may provide better adaptability and threat coverage in more dynamic or complex environments such as software-defined networks (SDNs), where temporal traffic patterns and evolving attack vectors prevail. Therefore, model selection should not be based solely on accuracy metrics but must also account for the operational context, including network topology, expected threat landscape, latency tolerance, and scalability requirements. This study advocates for a context-aware deployment strategy, where IDS model choice is aligned with the data structure and the real-world demands of the system in which it will operate.

Future research should explore the development of hybrid and ensemble models that integrate deep learning with classical machine learning algorithms to capitalize on the strengths of both approaches. Such combinations can improve detection accuracy and efficiency, particularly in multi-class classification tasks, where traditional models may excel in interpretability and deep models in learning complex patterns. Enhanced feature engineering techniques, such as the use of autoencoders or attention mechanisms for dimensionality reduction and representation learning, can also help reduce computational overhead without sacrificing performance. Moreover, implementing adaptive learning mechanisms is critical for enabling intrusion detection systems to respond to emerging threats, including zero-day attacks, by continuously updating model knowledge based on new data. Another promising direction is the incorporation of explainable AI (XAI) techniques to improve model transparency. XAI can provide actionable insights into model decisions, helping cybersecurity professionals understand, validate, and trust automated predictions, especially in high-stakes environments. Finally, evaluating these models across a broader range of heterogeneous and real-world datasets will further ensure robustness and generalizability in diverse cybersecurity contexts.

CRediT authorship contribution statement

Ayesha Alharthi: Visualization, Software, Formal analysis. **Meera Alaryani:** Visualization, Software, Formal analysis. **Sanaa Kaddoura:** Writing – review & editing, Writing – original draft, Visualization, Software, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

the data is cited in the paper

References

[1] Shahin M, Maghanaki M, Hosseinzadeh A, Chen FF. Advancing network security in industrial IoT: a deep dive into AI-enabled intrusion detection systems. Adv Eng Inform 2024;62:102685.
[2] Kaddoura S. Information matrix: fighting back using a matrix. In: 2015 IEEE/ACS 12th international conference of computer systems and applications (AICCSA); 2015. p. 1–6.
[3] Kaddoura S, Haraty RA, Al Kontar K, Alfandi O. A parallelized database damage assessment approach after cyberattack for healthcare systems. Future Internet 2021;13.
[4] Kaddoura Sanaa, Arid AE, M M. Evaluation of supervised machine learning algorithms for multi-class intrusion detection systems. In: Arai K, editor. Proceedings of the future technologies conference (FTC) 2021, vol. 3. Cham: Springer International Publishing; 2022. p. 1–16.
[5] Lal S, Taleb T, Dutta A. NFV: security threats and best practices. IEEE Commun Mag 2017;55:211–7.
[6] Kumar S, Gupta S, Arora S. Research trends in network-based intrusion detection systems: a review. IEEE Access 2021;9:157761–79.
[7] Nawaal B, Haider U, Khan IU, Fayaz M. Signature-based intrusion detection system for IoT. In: Cyber security for next-generation computing technologies. CRC Press; 2024. p. 141–58.
[8] Rakshe T, Gonjari V. Anomaly based network intrusion detection using machine learning techniques. Int J Eng Res Technol 2017;6:216–20.
[9] Aslan Ö, Aktuğ SS, Ozkan-Okay M, Yilmaz AA, Akin E. A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. Electronics 2023;12.
[10] Ozkan-Okay M, Akin E, Aslan Ö, Kosunalp S, Iliev T, Stoyanov I, Beloev I. A comprehensive survey: evaluating the efficiency of artificial intelligence and machine learning techniques on cyber security solutions. IEEE Access 2024;12:12229–56.
[11] Ahmad Z, Shahid Khan A, Wai Shiang C, Abdullah J, Ahmad F. Network intrusion detection system: a systematic study of machine learning and deep learning approaches. Transactions on Emerging Telecommunications Technologies 2021;32:e4150.
[12] Milosevic MS, Ciric VM. Extreme minority class detection in imbalanced data for network intrusion. Comput Secur 2022;123:102940.
[13] Milosevic M, Ciric V, Milentijevic I. Network intrusion detection using weighted voting ensemble deep learning model. In: 2024 11th international conference on electrical, electronic and computing engineering (IcETRAN); 2024. p. 1–6.
[14] Al-Janabi M, Ismail MA, Ali AH. Intrusion detection systems, issues, challenges, and needs. Int J Comput Intell Syst 2021;14:560–71.
[15] Kocher G, Kumar G. Machine learning and deep learning methods for intrusion detection systems: recent developments and challenges. Soft Comput 2021;25:9731–63.
[16] Yang Z, Liu X, Li T, Wu D, Wang J, Zhao Y, Han H. A systematic literature review of methods and datasets for anomaly-based network intrusion detection. Comput Secur 2022;116:102675.
[17] Lu C, Wang X, Yang A, Liu Y, Dong Z. A few-shot-based model-agnostic meta-learning for intrusion detection in security of internet of things. IEEE Internet Things J 2023;10(24):21309–21.
[18] Yang A, Lu C, Li J, Huang X, Ji T, Li X, Sheng Y. Application of meta-learning in cyberspace security: a survey. Digital Comm Networks 2023;9(1):67–78.
[19] Khan I, Khan J, Bangash SH, Ahmad W, Khan AI, hameed khalid. Intrusion detection using machine learning and deep learning models on cyber security attacks. VFAST Transactions on Software Engineering 2024;12:95–113.
[20] Cen M, Deng X, Jiang F, Doss R. Zero-Ran Sniff: a zero-day ransomware early detection method based on zero-shot learning. Comput Secur 2024;142:103849.
[21] Kale R, Thing VL. Few-shot weakly-supervised cybersecurity anomaly detection. Comput Secur 2023;130:103194.
[22] Ahuja N, Singal G, Mukhopadhyay D. DDOS attack SDN Dataset. 2020.
[23] Radoglou-Grammatikis P, Rompolos K, Sarigiannidis P, Argyriou V, Lagkas T, Sarigiannidis A, Goudos S, Wan S. Modeling, detecting, and mitigating threats against industrial healthcare systems: a combined software defined networking and reinforcement learning approach. IEEE Trans Ind Inf 2022;18:2041–52.
[24] Radoglou-Grammatikis P, Rompolos K, Lagkas T, Argyriou V, Sarigiannidis P. IEC 60870-5-104 intrusion detection dataset. 2022.
[25] Kumar V, Sinha D, Das AK, Pandey SC, Goswami RT. An integrated rule based intrusion detection system: analysis on UNSW-NB15 data set and the real time online dataset. Clust Comput 2020;23:1397–418.
[26] Díaz-Verdejo J, Muñoz-Calle J, Estepa Alonso A, Estepa Alonso R, Madinabeitia G. On the detection capabilities of signature-based intrusion detection systems in the context of web attacks. Appl Sci 2022;12.
[27] Ahmed O. Enhancing intrusion detection in wireless sensor networks through machine learning techniques and context awareness integration. International Journal of Mathematics, Statistics, and Computer Science 2024;2:244–58.
[28] Jeamaon A, Khemapatapan C. Development cyber risk assessment for intrusion detection using enhanced random forest. ECTI Transactions on Computer and Information Technology (ECTI-CIT) 2024;18:429–42.
[29] Mozo A, Karamchandani A, de la Cal L, Gómez-Canaval S, Pastor A, Gifre L. A machine-learning-based cyberattack detector for a cloud-based SDN controller. Appl Sci 2023;13.
[30] Mansoor A, Anbar M, Bahashwan AA, Alabsi BA, Rihan SDA. Deep learning-based approach for detecting DDoS attack on software-defined networking controller. Systems 2023;11.
[31] Tonkal Ö, Polat H, Başaran E, Cömert Z, Kocaoğlu R. Machine learning approach equipped with neighbourhood component analysis for DDoS attack detection in software-defined networking. Electronics 2021;10.
[32] Dhanya KA, Vajipayajula S, Srinivasan K, Tibrewal A, Kumar TS, Kumar TG. Detection of network attacks using machine learning and deep learning models. Procedia Comput Sci 2023;218:57–66.
[33] Aswathy MC, Rajkumar T. Real time anomaly detection in network traffic: a comparative analysis of machine learning algorithms. International Research Journal on Advanced Engineering Hub (IRJAEH) 2024;2:1968–77.
[34] European Union Agency for Network, Security, I.. Communication network dependencies for ICS/SCADA systems. 2017.
[35] Tesfahun A, Bhaskari DL. Intrusion detection using random forests classifier with SMOTE and feature reduction. In: 2013 international conference on cloud & ubiquitous computing & emerging technologies; 2013. p. 127–32.
[36] Guezzaz A, Benkirane S, Azrour M, Khurram S. A reliable network intrusion detection approach using decision tree with enhanced data quality. Secur Commun Network 2021;2021:1230593.
[37] Lakshminarayana SK, Basarkod PI. Unification of K-nearest neighbor (KNN) with distance aware algorithm for intrusion detection in evolving networks like IoT. Wirel Pers Commun 2023;132:2255–81.
[38] Chen Z, Jiang F, Cheng Y, Gu X, Liu W, Peng J. XGBoost classifier for DDoS attack detection and analysis in SDN-based cloud. In: 2018 IEEE international conference on big data and smart computing (BigComp); 2018. p. 251–6.
[39] Gouveia A, Correia M. Network intrusion detection with XGBoost. In: Recent advances in security, privacy, and trust for Internet of Things (IoT) and cyber-physical systems (CPS). Chapman and Hall/CRC; 2020. p. 137–66.
[40] Hnamte V, Hussain J. Dependable intrusion detection system using deep convolutional neural network: a Novel framework and performance evaluation approach. Telematics and Informatics Reports 2023;11:100077.
[41] Devendiran R, Turukmane AV. Dugat-LSTM: deep learning based network intrusion detection system using chaotic optimization strategy. Expert Syst Appl 2024;245:123027.
[42] Saurabh K, Sood S, Kumar PA, Singh U, Vyas R, Vyas OP, Khondoker R. LBDMIDS: LSTM based deep learning model for intrusion detection systems for IoT networks. In: 2022 IEEE world AI IoT congress (AIIoT); 2022. p. 753–9.
[43] Ge Y, Li J, Tian Y. Internet of things intrusion detection system based on D-GRU. In: 2022 4th international conference on applied machine learning (ICAML); 2022. p. 1–6.
[44] Modak A, Dehalwar V. Design and analysis of intrusion detection using deep learning models. In: 2024 IEEE international conference on electronics, computing and communication technologies (CONECCT); 2024. p. 1–6.
[45] Gamal M, Elhamahmy M, Taha S, Elmahdy H. Improving intrusion detection using LSTM-RNN to protect drones' networks. Egyptian Informatics Journal 2024;27:100501.